

**MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA**

**DESENVOLVIMENTO DE UMA APLICAÇÃO PARA SUPORTE AO
USO DE MEDICAMENTOS**

Daniel Welfer

Santa Maria, RS.
Março, 2026.

RESUMO

Este projeto tem como objeto o desenvolvimento de uma aplicação mobile destinada ao suporte no uso de medicamentos e especialmente voltada a pessoas idosas e/ou com dificuldades de leitura. O objetivo principal é desenvolver uma aplicação intuitiva, acessível e inclusiva, capaz de fornecer suporte visual e auditivo para facilitar a compreensão das informações relacionadas aos medicamentos, como horários, dosagens e identificação dos remédios. Busca-se, assim, reduzir erros na administração de medicamentos e tornar o acompanhamento do tratamento mais simples e confiável. A metodologia adotada consistirá no desenvolvimento incremental do sistema ao longo de um semestre acadêmico. Serão utilizadas ferramentas gratuitas de desenvolvimento, frameworks modernos e boas práticas de engenharia de software para a construção da aplicação. O projeto incluirá etapas de levantamento de requisitos, modelagem do sistema, implementação das funcionalidades principais e testes de usabilidade e funcionamento.

Como resultado esperado, pretende-se disponibilizar uma aplicação gratuita, leve e acessível, capaz de auxiliar usuários no gerenciamento de medicamentos por meio de uma interface simples e recursos de acessibilidade. Espera-se que a solução contribua para melhorar a organização do uso de medicamentos e promova maior inclusão digital no contexto da saúde.

Palavras-chave: *Aplicação móvel; Medicamentos; Saúde; Assistência médica; Inclusão digital.*

IDENTIFICAÇÃO

a) Tipo de ação de Extensão*:

As atividades a serem desenvolvidas enquadram-se na modalidade “Projeto de Extensão”.

b) Local de Execução:

O processo de desenvolvimento do sistema proposto será conduzido nas dependências da Universidade Federal de Santa Maria (UFSM), podendo também ocorrer de forma remota, conforme a necessidade da equipe. As etapas posteriores de validação e testes serão realizadas tanto nas dependências da UFSM quanto em clínicas parceiras, possibilitando tanto a avaliação da aplicação em contextos reais de uso, como possíveis adequações.

c) Período de execução:

As etapas de desenvolvimento e testes serão realizadas ao longo de um semestre letivo, com início em março e término em julho de 2026.

d) Equipe de trabalho:

<i>Nome</i>	<i>Vínculo</i>	<i>Função</i>
<i>Filipe Sacchet Kaizer</i>	<i>Estudante do Curso de Sistemas de Informação</i>	<i>Desenvolvedor</i>
<i>Daniel Welfer</i>	<i>Docente</i>	<i>Orientador</i>

e) Público:

O sistema a ser desenvolvido terá como foco pessoas com dificuldades de leitura e usuários que necessitam auxílio no controle de medicamentos.

INTRODUÇÃO

O presente projeto de extensão propõe o desenvolvimento de uma aplicação mobile voltada ao suporte no uso de medicamentos, com foco especial em pessoas idosas e indivíduos com dificuldades de leitura ou organização no gerenciamento de tratamentos. A aplicação terá como principal finalidade auxiliar os usuários no controle dos horários, dosagens e identificação dos medicamentos, contribuindo para a redução de erros na administração e promovendo maior autonomia e segurança no cuidado com a saúde.

OBJETIVOS

OBJETIVO GERAL

Desenvolver uma aplicação mobile acessível para auxiliar no gerenciamento e uso correto de medicamentos.

OBJETIVOS ESPECÍFICOS

Destacam-se para este projeto os seguintes pontos:

- Elaborar uma base de dados relacional com todos os medicamentos disponibilizados pelo sistema público de saúde com suas respectivas dosagens, classificações e imagens;
- Criar uma aplicação mobile acessível e com interfaces intuitivas;
- Implementar um sistema de notificação de horários;
- Adicionar funcionalidades de gamificação;
- Adicionar sistemas inclusivos, como voz e adaptação de texto;
- Realizar testes com usuários reais.

JUSTIFICATIVA

O presente projeto de extensão justifica-se pela necessidade de auxiliar a população no uso correto de medicamentos, especialmente pessoas idosas e indivíduos com dificuldades de leitura ou organização de rotinas terapêuticas. O uso inadequado de medicamentos pode comprometer tratamentos e gerar riscos à saúde, tornando relevante o desenvolvimento de soluções tecnológicas acessíveis que promovam maior segurança e autonomia aos usuários. A proposta está alinhada às diretrizes da extensão universitária da UFSM ao promover a interação dialógica entre universidade e sociedade, partindo de uma demanda real e buscando solucioná-la por meio do desenvolvimento de uma aplicação voltada ao público externo. A realização de testes em contextos reais contribui para aproximar a comunidade do ambiente acadêmico, permitindo a construção conjunta do conhecimento.

No que diz respeito à formação do estudante, o projeto possibilita a aplicação prática de conhecimentos adquiridos no curso de Sistemas de Informação, integrando aspectos como desenvolvimento de software, banco de dados e usabilidade. Além disso, favorece o desenvolvimento do senso crítico e da responsabilidade social.

Por fim, o projeto demonstra comprometimento com as demandas da sociedade ao propor uma solução inclusiva e de impacto na área da saúde, contribuindo para a redução de erros no uso de medicamentos e para a promoção da inclusão digital. Dessa forma, reforça o papel da universidade pública como agente de transformação social.

REFERENCIAIS TEÓRICOS E CONCEITUAIS

(identificar e comentar sobre conhecimentos já produzidos, em relação ao objeto da ação extensionista, adequados à proposta a ser efetivada, além do aporte de ideias e percepções de diferentes autores que se dedicaram ao tema, e também informações sobre documentação empírica que contribui para caracterizar o entendimento do objeto da ação)

METODOLOGIA DA AÇÃO

O processo de desenvolvimento será estruturado em três etapas principais: levantamento de requisitos; modelagem da base de dados, desenvolvimento da API (*Application Programming Interface*) central e desenvolvimento da aplicação *mobile*.

a) Levantamento de Requisitos

O levantamento de requisitos foi realizado com base na proposta de desenvolvimento de uma aplicação *mobile* destinada ao suporte no uso de medicamentos, considerando as necessidades de usuários que demandam auxílio na organização e acompanhamento de tratamentos. Os requisitos foram classificados em requisitos funcionais e não funcionais, visando garantir clareza na definição das funcionalidades e das características esperadas do sistema.

Para facilitar a compreensão da estrutura do sistema, o mesmo será composto por três camadas principais: aplicação *mobile* (cliente), API (backend) e banco de dados (armazenamento de remédios).

APLICAÇÃO MOBILE

Quanto aos requisitos funcionais, a aplicação *mobile* deverá:

- Exibir informações detalhadas dos medicamentos presentes na base de dados (nome, dosagem, horários, classificação e imagens);
- Permitir o cadastro de horários de consumo de remédios de forma manual ou automática (com base no horário inicial e na periodicidade);
- Exibir notificações automáticas e lembretes de horários programados;
- Oferecer opções de assistência como leitura de tela e adaptação de interface;
- Permitir ao usuário a confirmação da ingestão do medicamento;
- Exibir uma lista de medicamentos detalhada e organizada de forma clara;

- Possuir sincronia automática com a API central (*backend*);
- Armazenar informações dos remédios utilizados e de doses a serem tomadas na memória cache do sistema;
- Registrar o cumprimento das atividades;
- Atribuir pontos ao usuário para cada ação concluída corretamente;
- Exibir um progresso diário e/ou mensal;
- Implementar sistema de metas, como dias consecutivos sem falhas;
- Implementar um sistema de conquistas, com animações, mensagens motivacionais, níveis e medalhas digitais;
- Implementar sistemas de acessibilidade, como leitura de tela e texto com tamanho adaptável às necessidades do usuário.

Quanto aos requisitos não funcionais, a aplicação *mobile* deve:

- Possuir interface simples, intuitiva e acessível;
- Uso de paleta de cores e textos chamativos e confortáveis para idosos;
- Baixo consumo do armazenamento e processamento do sistema;
- Tempo de resposta rápido, inclusive em dispositivos antigos;
- Compatibilidades com diferentes dispositivos *Android*;
- Usabilidade acessível, sem funcionalidades complexas.

API CENTRAL

A API central compreende a interface de comunicação e tratamento de dados existente entre o dispositivos *mobile* e o banco de dados. Desta forma, quanto aos requisitos funcionais, a mesma deverá:

- Fornecer rotas para obter informações sobre um medicamento específico (com base no seu *id*), informações sobre todos os medicamentos armazenados e imagens de um medicamento específico (com base no seu *id*);
- Tratar informações oriundas do banco de dados, retornando-as à aplicação no formato *JavaScript Object Notation (JSON)*;
- Realizar o tratamento das imagens obtidas de um diretório;
- Verificar utilizando algoritmos de inteligência artificial se a imagem é de um

remédio.

Quanto aos requisitos não funcionais, a API deve:

- Ser implementada sob a arquitetura REST (*Representational State Transfer*) e MVC (*Model-View-Controller*);
- Ter alta disponibilidade, podendo atender pelo menos 100 usuário simultâneos;
- Validar os dados de entrada, impedindo falhas de segurança na base de dados;
- Responder as requisições em no máximo 500ms;
- Ser bem estruturada, segundo os princípios da orientação a objetos.

BANCO DE DADOS

O banco de dados será responsável pelo armazenamento de medicamentos e a referência das suas respectivas imagens associadas. Desta forma, quanto aos requisitos funcionais, o mesmo deverá:

- Armazenar informações de medicamentos (*id*, nome, tipo, doses);
- Armazenar diferentes dosagens associadas aos medicamentos;
- Permitir relacionamento muitos-para-muitos entre os medicamentos e as dosagens;
- Armazenar o caminho para as imagens junto a identificação do seu respectivo medicamento associado;
- Garantir que cada imagem seja associada a apenas um medicamento;

Quanto aos requisitos não funcionais, o banco de dados deve:

- Utilizar um Sistema de Gerenciamento de Banco de Dados (SGBD) gratuito e relacional (MySQL);
- Possuir estrutura normalizada;
- Possuir acesso restrito apenas para a API central.

b) Base de Dados

Para o desenvolvimento do sistema, foi utilizada uma lista de medicamentos disponibilizada pela base de dados do Departamento de Assistência Farmacêutica (FAMURS, 2026) e a Secretaria de Saúde do Rio Grande do Sul (Secretaria da Saúde,

2025), totalizando 256 remédios.

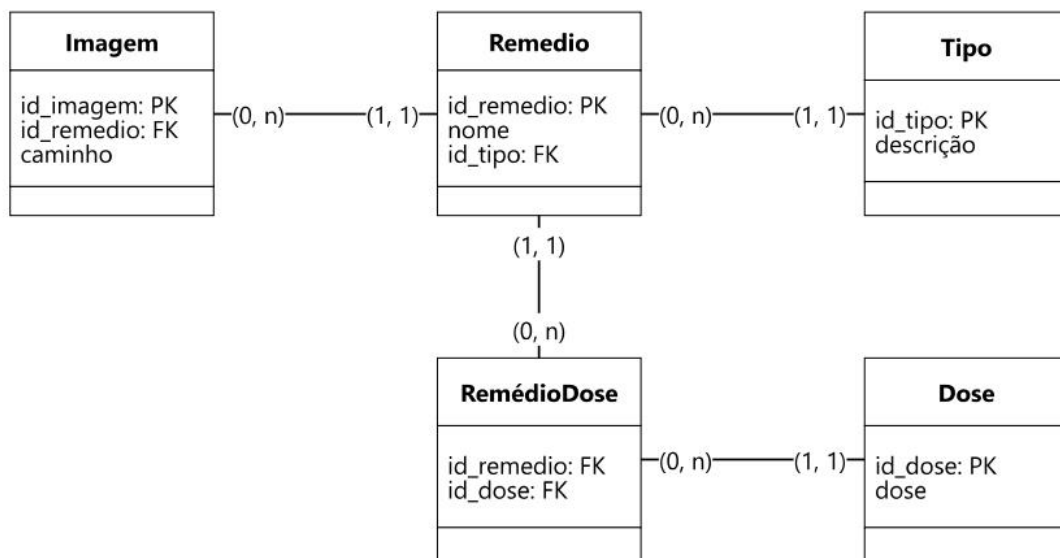
Conforme observado nessa base, cada medicamento possui atributos como nome, dosagem, forma de uso (oral ou injetável) e tipo (genérico ou de referência).

Um mesmo medicamento pode apresentar diferentes dosagens, assim como uma mesma dosagem pode estar associada a múltiplos medicamentos, caracterizando um relacionamento do tipo muitos-para-muitos. Além disso, cada medicamento pode possuir uma ou mais imagens associadas, enquanto cada imagem está vinculada exclusivamente a um único medicamento.

As imagens são armazenadas em um diretório específico no sistema, sendo registrado no banco de dados apenas o caminho de acesso ao arquivo, juntamente com a identificação do medicamento ao qual a imagem está associada. É importante que todas as tabelas criadas possuam um campo de identificação numérico e incremental, para facilitar futuras indexações.

Com base nesses aspectos, foi desenvolvido o modelo entidade-relacionamento apresentado na Figura 1.

Figura 1 - Modelo Entidade-Relacionamento



Fonte: Autor, 2026.

Adicionalmente, é importante que a aplicação possua um sistema de pontuação. Desta forma, os usuários poderão verificar o seu histórico de interações com a aplicação e também a pontuação obtida, inspirando os usuários a tomar seus medicamentos no período

correto.

Para garantir a segurança dos usuários, as pontuações e horários só poderão ser armazenadas no dispositivo móvel, sem nenhuma relação com a base de dados.

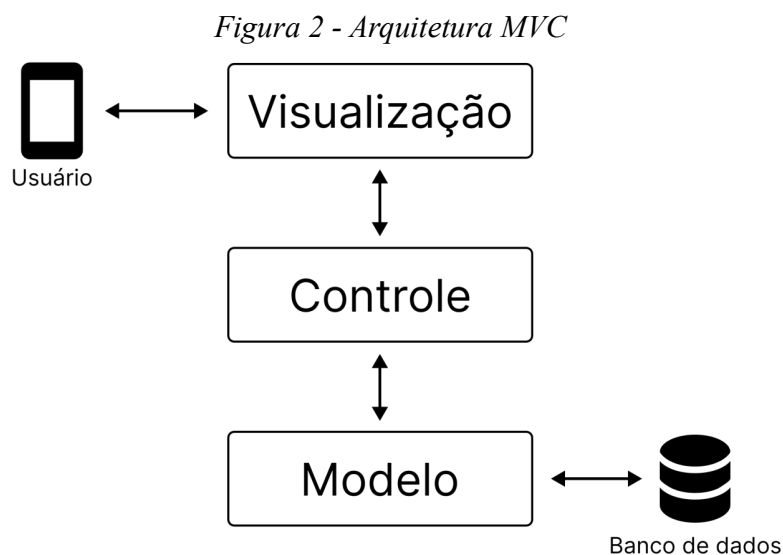
c) API Central

Por ser de grande importância para o funcionamento da aplicação móvel, esta seção será dividida em uma definição de arquitetura, desenvolvimento da inteligência artificial (IA) para a verificação das imagens e os testes de desempenho.

ARQUITETURA

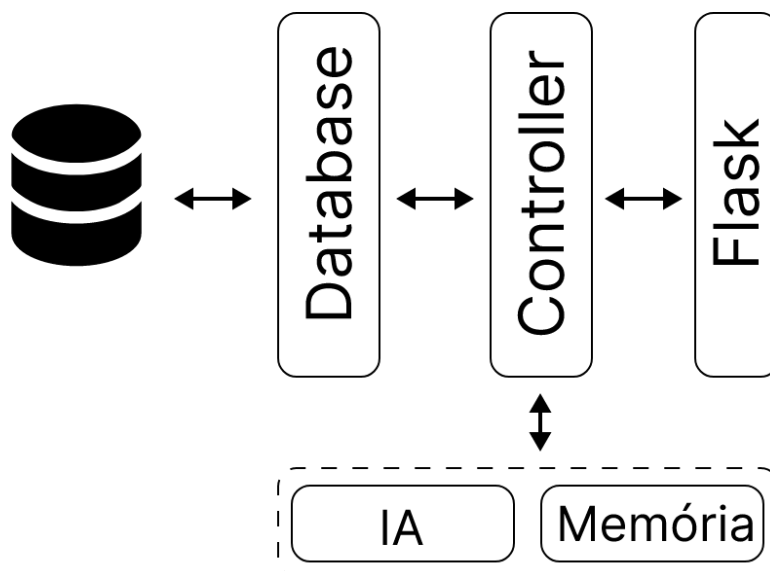
Visando o correto tratamento e gerenciamento dos dados provenientes do banco de dados, foi desenvolvida uma API REST baseada na arquitetura MVC. Essa arquitetura organiza o sistema em três camadas principais: o Modelo (*Model*), responsável pela lógica de negócio e manipulação dos dados; a Visualização (*View*), responsável pela interface de comunicação com o cliente da aplicação; e o Controle (*Controller*), que atua como intermediário entre as requisições recebidas e o processamento das regras de negócio, conforme ilustrado na Figura 2.

Para o desenvolvimento da API, será utilizada a linguagem de programação *Python*, em conjunto com o *framework Flask*, responsável por implementar a camada de comunicação (rotas e *endpoints*). A arquitetura final da API é apresentada na Figura 3.



Fonte: Autor, 2026.

Figura 3 - Arquitetura da API



Fonte: Autor, 2026.

A arquitetura da API foi projetada para suportar múltiplos acessos simultâneos, reduzindo a necessidade de consultas recorrentes à base de dados. Para a camada de comunicação com os usuários, responsável pela disponibilização das informações e interação com a aplicação, foi utilizado o framework Flask, no qual foram implementadas as rotas para obtenção de dados, envio de imagens e consulta de medicamentos.

Com o objetivo de otimizar o desempenho das operações de leitura, foi desenvolvida uma camada de memória centralizada (Memória), responsável pelo armazenamento temporário e acesso rápido aos dados dos medicamentos, evitando consultas desnecessárias ao banco de dados durante a operação da aplicação. Essa camada também gerencia uma fila de processamento de imagens, permitindo que o envio e a validação das imagens ocorram de forma assíncrona, sem bloquear as requisições dos usuários.

O acesso à base de dados foi encapsulado pelo serviço *Database*, responsável por concentrar as operações de consulta, inserção e atualização das informações persistidas. Essa separação permite maior organização do código, isolamento das operações de persistência e maior segurança no gerenciamento dos dados.

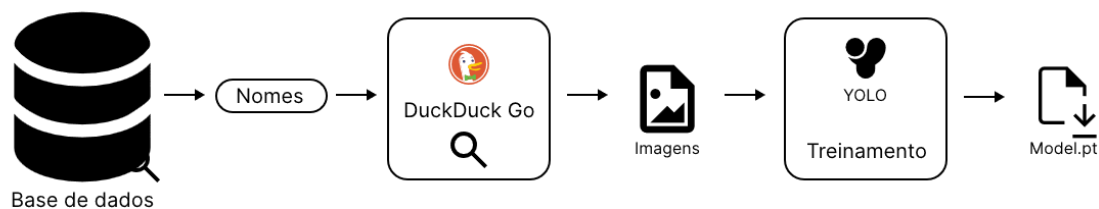
Além dos serviços responsáveis pelo gerenciamento dos dados, foi incorporado um módulo de validação de imagens baseado em IA. Esse serviço utiliza um modelo de visão computacional para analisar as imagens enviadas pelos usuários, verificando sua

compatibilidade com as classes de medicamentos esperadas antes do armazenamento definitivo das informações.

INTELIGÊNCIA ARTIFICIAL

O algoritmo de IA será desenvolvido e treinado pelo desenvolvedor utilizando um conjunto de imagens obtidas a partir de ferramentas de busca na internet. A Figura 4 apresenta o fluxo do processo de treinamento do modelo de IA, iniciando pela consulta dos nomes dos medicamentos disponíveis na base de dados. Posteriormente, são realizadas buscas por imagens correspondentes utilizando o mecanismo de pesquisa *DuckDuckGo*, formando o conjunto de dados utilizado no treinamento. Por fim, as imagens coletadas são utilizadas para o treinamento do modelo de detecção de objetos baseado na arquitetura *YOLO*, que será empregado na identificação de medicamentos presentes nas imagens submetidas pelos usuários.

Figura 4 - Treinamento da IA



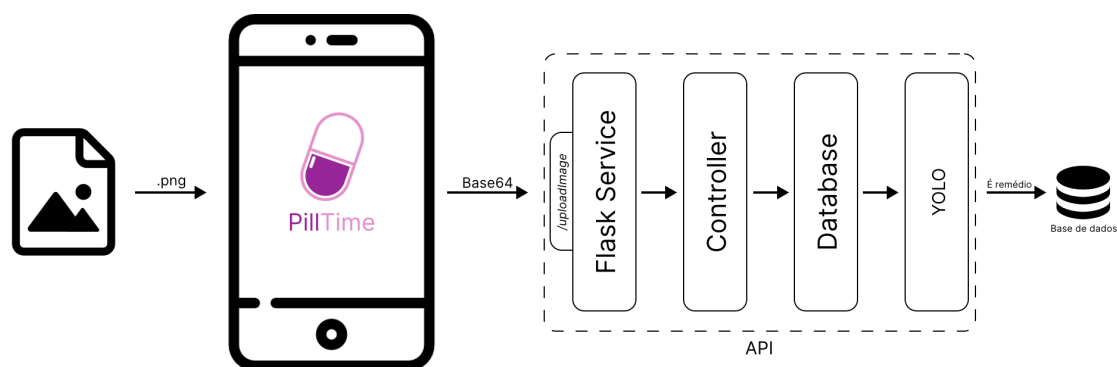
Fonte: Autor, 2026.

Para o treinamento do modelo de IA, foram utilizadas 1.216 imagens contendo medicamentos e 80 imagens sem a presença de medicamentos, totalizando 1.296 amostras. O conjunto de dados foi dividido igualmente, sendo 50% das imagens destinadas ao treinamento do modelo e 50% destinadas à etapa de validação, permitindo avaliar sua capacidade de generalização diante de dados não utilizados durante o aprendizado. O processo de treinamento foi realizado utilizando 50 épocas com uma placa de vídeo NVIDIA RTX 3050 como unidade de processamento principal.

O modelo obtido durante o processo de treinamento é integrado ao serviço *Flask_service*, sendo utilizado especificamente pela rota *uploadImage*, responsável pelo recebimento, processamento e validação das imagens enviadas pela aplicação móvel. Na Figura 5 é

apresentado o fluxo de envio, validação e armazenamento das imagens na base de dados. Durante esse processo, a imagem selecionada pelo usuário é inicialmente convertida pela aplicação móvel para o formato *Base64* e encaminhada à API através da rota *uploadImage*. Ao receber a requisição, o serviço realiza a decodificação da imagem e a submete ao modelo de inteligência artificial previamente treinado *YOLO*, responsável por avaliar sua compatibilidade com uma imagem de medicamento. Caso a validação seja aprovada, a imagem é armazenada na base de dados e associada ao respectivo registro do medicamento. Todo esse processo é executado de forma transparente ao usuário, sem a necessidade de interações adicionais durante a validação.

Figura 5 - Envio de imagem com verificação da IA



Fonte: Autor, 2026.

TESTES DE DESEMPENHO

Para validar a arquitetura desenvolvida, foi implementado um algoritmo de testes de acesso simultâneo, com o objetivo de avaliar a capacidade de atendimento da API sob diferentes níveis de carga. Foram simulados até 100 acessos simultâneos, considerando um total de 500 requisições realizadas por clientes distintos. Durante os testes, foram executadas operações de consulta geral de medicamentos, pesquisa de um medicamento específico e envio de imagens para processamento. Para a avaliação do desempenho da arquitetura, foram consideradas as métricas de quantidade de acessos simultâneos suportados e latência média das requisições. Na Tabela 1 são apresentadas as rotas utilizadas nos testes, juntamente com suas respectivas descrições, métodos HTTP e parâmetros enviados em cada requisição.

Tabela 1 - Rotas testadas

Rota	Método	Descrição	Parâmetro
/getRemedies	GET	Retorna a lista de medicamentos disponíveis no sistema, incluindo as informações cadastradas na base de dados.	-
/image	GET	Obtém as imagens associadas a um medicamento específico. As imagens são carregadas do armazenamento e convertidas para Base64 para envio através da API.	• <i>remedy</i> : ID do remédio;
/images/<filename>	GET	Disponibiliza diretamente um arquivo de imagem armazenado no servidor. Utilizada quando é necessário obter a imagem em seu formato original.	• <i>filename</i> : nome do arquivo da imagem armazenada.
/uploadImage	POST	Realiza o envio de uma imagem de medicamento para a API. A imagem é encapsulada em um objeto <i>image</i> e adicionada à fila de processamento assíncrono para validação por inteligência artificial	• <i>remedy</i> : ID do remédio; • <i>image</i> : arquivo de imagem enviado pelo usuário.

Fonte: Autor, 2026.

Para a execução dos testes envolvendo as rotas /uploadImage e /image, foi utilizado o medicamento identificado pelo ID 107, correspondente à Cabergolina. Para as operações de consulta e envio de imagens, realizadas pelas rotas /images/<filename> e /uploadImage, foi utilizada a imagem apresentada na Figura 6. O arquivo utilizado possui resolução de 3840×2160 pixels (4K) e tamanho de 945 KB, permitindo avaliar o comportamento da API durante o processamento e transferência de arquivos de maior dimensão. Além disso, todos os testes foram realizados no mesmo dispositivo, eliminando a latência da rede.

Figura 6 - Imagem de testes (3840 x 2160 - 4K)



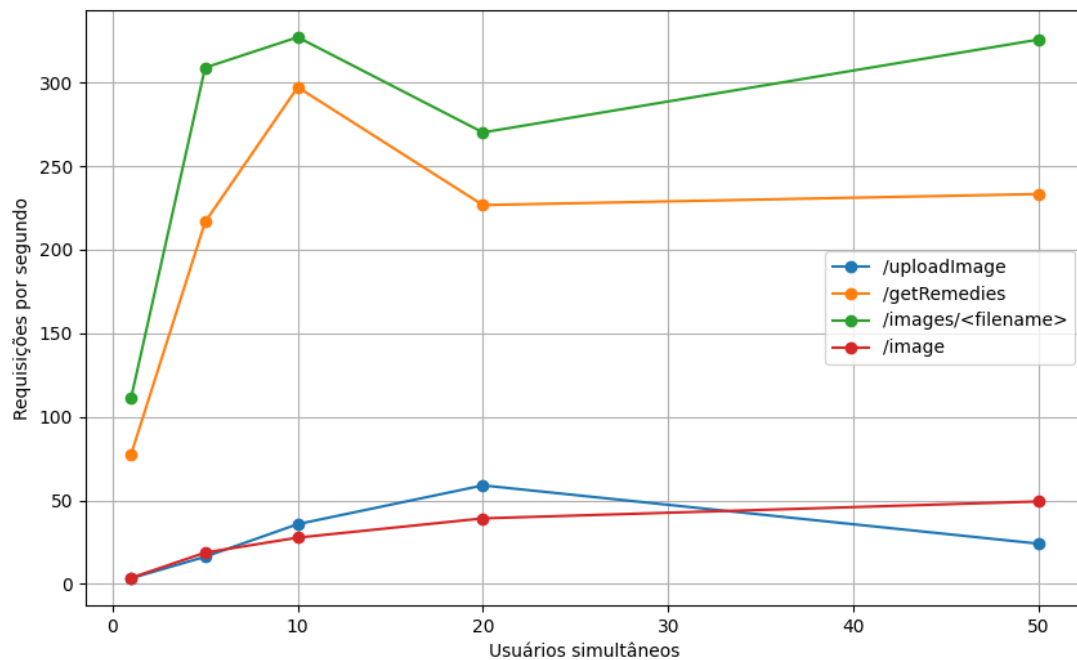
Fonte: Autor, 2026.

Na Figura 7 são apresentadas as capacidades de atendimento simultâneo obtidas para cada rota avaliada. Observa-se que as rotas responsáveis pela obtenção da lista completa de medicamentos e pela disponibilização direta de imagens (*/getRemedies* e */images/<filename>*, respectivamente) apresentaram os melhores resultados, suportando mais de 300 requisições por segundo. Esse desempenho está relacionado ao baixo custo computacional dessas operações, uma vez que envolvem consultas apenas à memória da API.

Em contrapartida, as rotas responsáveis pelo envio e recuperação de imagens (*/uploadImage* e */image*, respectivamente) apresentaram capacidade inferior, com valores pouco superiores a 50 requisições por segundo. Essa redução de desempenho é esperada, pois essas operações envolvem transferência de arquivos, operações de leitura e escrita em disco e, no caso da consulta de imagens, a conversão dos arquivos para o formato Base64 antes do envio ao cliente. Apesar da menor capacidade quando comparadas às demais rotas, os resultados obtidos para as operações envolvendo imagens atendem aos requisitos estabelecidos para a API, que definem uma capacidade mínima de 50 requisições simultâneas por segundo. Dessa forma, os testes demonstram que a arquitetura

desenvolvida consegue manter o atendimento das operações críticas mesmo sob cenários de concorrência elevada.

Figura 7 - Resultados para a métrica de requisições por segundo

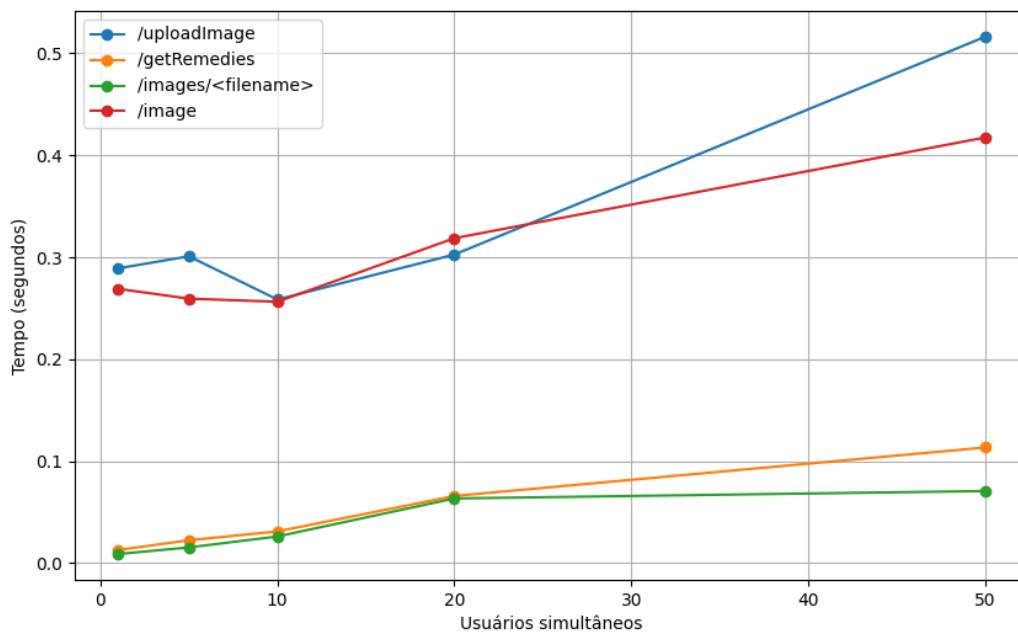


Fonte: Autor, 2026.

Estas análises também se refletem no tempo de resposta para cada rota, apresentado na Figura 8. Observa-se que a maioria das rotas avaliadas apresentou latências inferiores a 500 ms, valor estabelecido como requisito para o funcionamento adequado da API. Assim como observado no teste de capacidade de requisições simultâneas, as rotas relacionadas ao envio e à consulta de imagens apresentaram os maiores tempos de resposta, com latências superiores a 200 ms, devido ao maior volume de dados processados e à necessidade de operações de leitura e escrita de arquivos.

Por outro lado, as demais rotas responsáveis pela consulta de informações dos medicamentos apresentaram tempos de resposta inferiores a 100 ms. Esse resultado está associado à utilização de mecanismos de armazenamento em memória, que reduzem a necessidade de acessos frequentes à fonte de dados original e, consequentemente, diminuem o tempo necessário para o processamento das requisições.

Figura 8 - Resultados para a métrica latência



Fonte: Autor, 2026.

Com base nos resultados obtidos, foi possível determinar que a API atende aos requisitos estabelecidos no início do seu desenvolvimento. A aplicação apresentou uma capacidade de processamento superior a 50 requisições simultâneas, atingindo valores superiores a 300 requisições por segundo em determinadas rotas, além de apresentar latências inferiores ao limite de 500 ms estabelecido para a maioria das operações. As rotas relacionadas ao processamento de imagens apresentaram tempos de resposta próximos ou pouco superior a esse limite, enquanto as rotas responsáveis pela consulta de dados gerais mantiveram latências inferiores a 100 ms.

d) Aplicação móvel

O processo de desenvolvimento da aplicação mobile será dividido em três etapas principais: escolha de tecnologias, modelagem lógica e design.

ESCOLHA DE TECNOLOGIAS

Para o desenvolvimento da aplicação, foram avaliadas diferentes abordagens de desenvolvimento mobile, considerando aspectos como desempenho, compatibilidade entre

plataformas, facilidade de manutenção e disponibilidade de recursos. As principais alternativas analisadas foram o desenvolvimento nativo, utilizando os SDKs (*Software Development Kits*) oficiais de cada sistema operacional, e os frameworks multiplataforma *Flutter*, *React Native* e *Xamarin/.NET MAUI*.

A Tabela 2 apresenta uma comparação entre as principais tecnologias consideradas.

Tabela 2 - Tecnologias de desenvolvimento

Framework	Linguagem	Plataformas	Vantagens	Desvantagens
Desenvolvimento Nativo	<i>Kotlin/Java (Android)</i> <i>Swift/Objective-C (iOS)</i>	<i>Android</i> e <i>iOS</i>	• Alto desempenho • Acesso à recursos do sistema; • Integração completa com o sistema.	• Códigos separados para cada plataforma; • <i>Designs</i> diferentes.
<i>React Native</i>	<i>JavaScript</i> <i>TypeScript</i>	<i>Android</i> , <i>iOS</i> e <i>Web</i>	• Baseado na comunicação entre código <i>JavaScript</i> e componentes nativos.	• Comunidade ampla; • Grande quantidade de bibliotecas.
<i>Xamarin/.NET MAUI</i>	C#	<i>Android</i> , <i>iOS</i> , <i>Windows</i> e <i>macOS</i>	• <i>Framework</i> integrado ao ecossistema .NET.	• Maior dependência do ecossistema .NET; • Menor adoção em novos projetos <i>mobile</i>.
<i>Flutter</i>	<i>Dart</i>	<i>Android</i> , <i>iOS</i> , <i>Web</i> , <i>Windows</i> , <i>macOS</i> e <i>Linux</i>	• <i>Framework</i> desenvolvido e mantido pela <i>Google</i>; • Mecanismo próprio de renderização de interface; • <i>Design</i> idêntico em diferentes plataformas; • Ampla comunidade e bibliotecas.	• Necessidade de aprendizado da linguagem <i>Dart</i>.

Fonte: Autor, 2026.

O desenvolvimento nativo apresenta como principal vantagem o alto desempenho e o acesso direto aos recursos disponibilizados pelo sistema operacional. Entretanto, essa abordagem exige a implementação de versões independentes para cada plataforma,

aumentando a complexidade de manutenção e o tempo necessário para evolução da aplicação.

Entre as soluções multiplataforma, o *React Native*, desenvolvido pela *Meta*, utiliza *JavaScript* e permite a criação de interfaces utilizando componentes nativos dos sistemas operacionais. Apesar de possuir ampla adoção e uma grande quantidade de bibliotecas disponíveis, sua arquitetura depende da comunicação entre o código *JavaScript* e os componentes nativos, podendo apresentar limitações em aplicações que demandam maior interação com recursos do dispositivo.

O *Xamarin*, posteriormente substituído pelo .NET MAUI, utiliza a linguagem C# e permite o compartilhamento de código entre diferentes plataformas. Embora apresente integração com o ecossistema .NET, sua utilização no desenvolvimento de novas aplicações mobile é menor quando comparada a *frameworks* mais recentes

O *Flutter*, desenvolvido pela *Google*, utiliza a linguagem *Dart* e possui um mecanismo próprio de renderização de interfaces, permitindo maior controle visual e desempenho consistente entre diferentes plataformas. Diferentemente de soluções baseadas em componentes nativos, o *Flutter* reduz a dependência das APIs específicas de cada sistema operacional, possibilitando o desenvolvimento de aplicações *Android*, iOS e outras plataformas utilizando uma única base de código.

Dessa forma, o *Flutter* foi escolhido para o desenvolvimento da aplicação *Pill Time* devido à sua arquitetura moderna, capacidade multiplataforma e eficiência no desenvolvimento de aplicações *mobile*. A possibilidade de reutilização do código entre diferentes sistemas operacionais reduz a complexidade de manutenção, enquanto seu desempenho e suporte a recursos nativos atendem aos requisitos da aplicação, como notificações, armazenamento local, gerenciamento de imagens e interação com os recursos do dispositivo

Além disso, foram utilizadas diversas bibliotecas disponíveis no ecossistema *Flutter* para adicionar funcionalidades específicas, como gerenciamento de estado, armazenamento local, notificações, manipulação de imagens e geração de documentos. A Tabela 3 apresenta as principais bibliotecas utilizadas e suas respectivas finalidades.

Tabela 3 - Bibliotecas utilizadas

Biblioteca	Descrição
<i>Provider</i>	Responsável pelo gerenciamento de estado da aplicação, permitindo compartilhamento de dados entre diferentes componentes da interface.
<i>flutter_local_notifications</i>	Implementa o sistema de notificações locais utilizado para alertar os usuários sobre os horários dos medicamentos
<i>hive_flutter</i>	Implementa armazenamento local rápido utilizando o banco de dados <i>Hive</i> integrado ao Flutter
<i>flutter_tts</i>	Realiza a conversão de texto em fala, permitindo recursos de interação por áudio
<i>pdf</i>	Permite a criação de arquivos PDF dentro da aplicação
<i>printing</i>	Disponibiliza recursos de visualização e impressão de documentos gerados pela aplicação

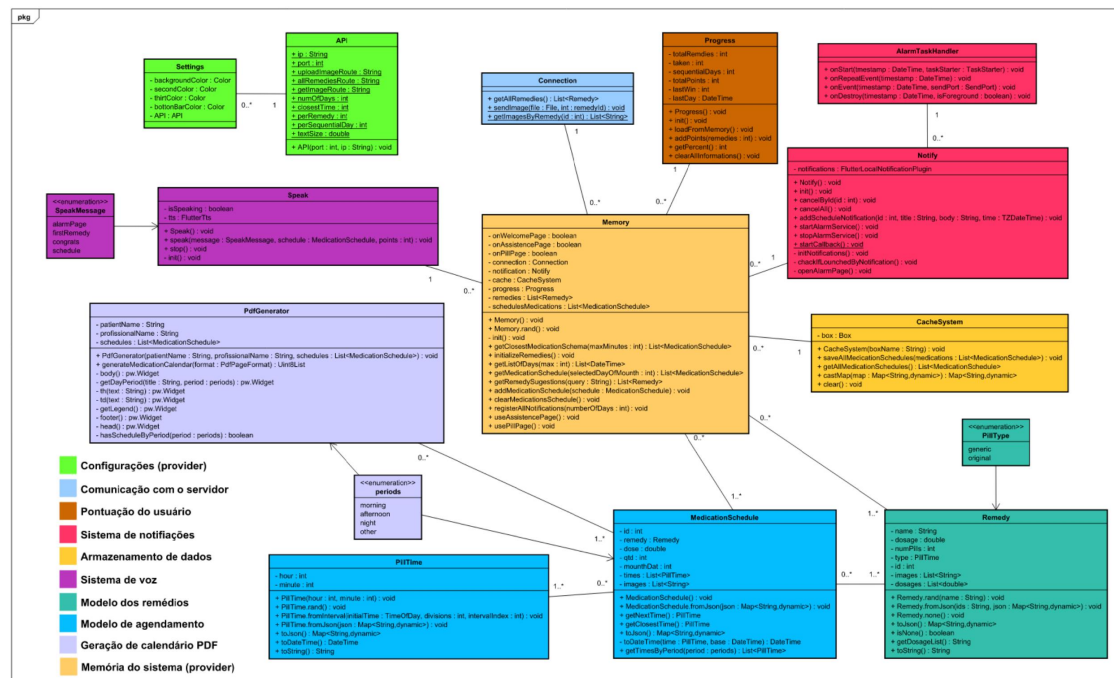
Fonte: Autor, 2026.

Além das bibliotecas externas, a aplicação utiliza recursos nativos disponibilizados pelo próprio *Flutter*, como o conjunto de componentes visuais baseado em *Material Design*, gerenciamento de *assets* e ferramentas de testes automatizados. A combinação dessas bibliotecas possibilitou o desenvolvimento dos principais módulos da aplicação, incluindo gerenciamento dos medicamentos, criação de alarmes, armazenamento local, geração de relatórios e interação com o usuário.

ESCOLHA DE TECNOLOGIAS

Na figura x é apresentado o diagrama de classes desenvolvido para a aplicação. As classes foram organizadas conforme suas responsabilidades, permitindo identificar os módulos de configuração, comunicação externa, gerenciamento de dados, controle de medicamentos, notificações e funcionalidades auxiliares.

Figura 9 - Diagrama de classes da aplicação



Fonte: Autor, 2026.

O núcleo da aplicação é representado pela classe *Memory*, responsável pelo gerenciamento do estado da aplicação e pela integração entre os diferentes componentes. Essa classe mantém referências para informações carregadas durante a execução, como medicamentos cadastrados, agendamentos, configurações do usuário, sistema de notificações, cache local e informações de progresso. Além disso, disponibiliza métodos responsáveis pela inicialização dos dados, recuperação de medicamentos, gerenciamento dos horários de uso e controle das informações armazenadas.

A comunicação com serviços externos é realizada pelo conjunto formado pela classe *Connection* com os dados obtidos das classes *Settings* e *API*. A classe *API* concentra as configurações necessárias para conexão com o servidor, como endereço, porta e rotas disponíveis, enquanto a classe *Connection* abstrai as requisições realizadas pela aplicação, permitindo obter a lista de medicamentos, consultar imagens associadas e recuperar informações específicas de um medicamento.

O gerenciamento dos medicamentos é realizado principalmente pelas classes *Remedy* e *MedicationSchedule*. A classe *Remedy* representa o modelo de um medicamento,

armazenando informações como nome, dosagem, quantidade de comprimidos, imagens e tipos disponíveis. Já a classe *MedicationSchedule* representa o planejamento de utilização do medicamento, contendo informações relacionadas à dose, quantidade, período de administração, horários e imagens exibidas durante os alarmes. Dessa forma, um medicamento pode possuir diferentes agendamentos associados, permitindo representar múltiplas rotinas de uso.

A classe *PillTime* representa a estrutura responsável pelo armazenamento dos horários individuais de administração dos medicamentos. Ela contém informações de hora e minuto e possui métodos para conversão e manipulação desses valores, sendo utilizada pela classe *MedicationSchedule* para definir os momentos em que os medicamentos devem ser administrados. O sistema de notificações é representado pelas classes *Notify* e *AlarmTaskHandler*. A classe *Notify* é responsável pela criação, gerenciamento e cancelamento das notificações locais utilizando o serviço de notificações do dispositivo. Já a classe *AlarmTaskHandler* controla os eventos associados aos alarmes, definindo ações executadas no início, repetição e encerramento de uma notificação.

O controle da pontuação e acompanhamento do usuário é realizado pela classe *Progress*, responsável por armazenar informações como pontos acumulados, sequência de dias de utilização, último registro de uso e progresso geral. Adicionalmente, para otimizar o acesso aos dados, a aplicação utiliza a classe *CacheSystem*, responsável pelo armazenamento local das informações obtidas. Essa classe permite salvar e recuperar medicamentos e agendamentos, reduzindo a necessidade de consultas frequentes ao servidor e contribuindo para menores tempos de resposta durante a utilização do aplicativo.

A funcionalidade de geração de documentos é implementada pela classe *PdfGenerator*, responsável pela criação do calendário de medicamentos em formato PDF. Essa classe recebe as informações do paciente, profissional responsável e agendamentos cadastrados, gerando um documento contendo a programação dos medicamentos.

Por fim, as classes *Settings* e *Speak* representam funcionalidades auxiliares da aplicação. A classe *Settings* mantém as configurações visuais e parâmetros de funcionamento do aplicativo, como cores da interface e endereço da API. Já a classe *Speak* implementa o

sistema de síntese de voz, permitindo a reprodução de mensagens relacionadas aos medicamentos e alarmes.

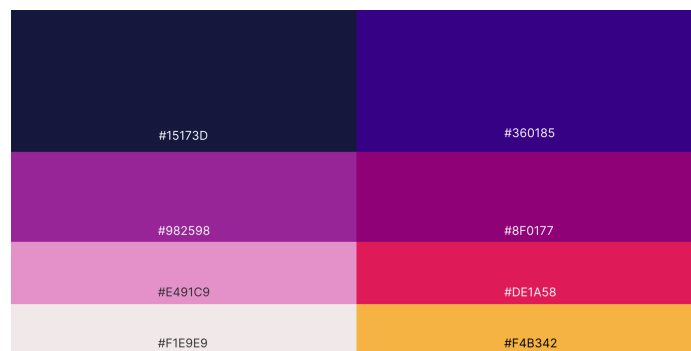
As enumerações presentes no diagrama, como *PillType*, *periods* e *SpeakMessage*, representam conjuntos de valores predefinidos utilizados para padronizar informações dentro da aplicação.

De maneira geral, o diagrama demonstra uma arquitetura modular, em que as responsabilidades são distribuídas entre diferentes classes, facilitando a manutenção, expansão e compreensão do código. A separação entre modelos de dados, serviços externos, armazenamento local e funcionalidades de interação com o usuário contribui para uma maior organização e escalabilidade da aplicação.

DESIGN

Por se tratar de uma aplicação voltada a idosos, foram escolhidas paletas de cores com tons mais quentes e com cores distintas, visando a delimitação das diferentes partes do sistema (fundo, botões e texto) de forma clara. As cores escolhidas são apresentadas na Figura 10, com seus respectivos códigos de referência em hexadecimal.

Figura 10 - Paleta de cores

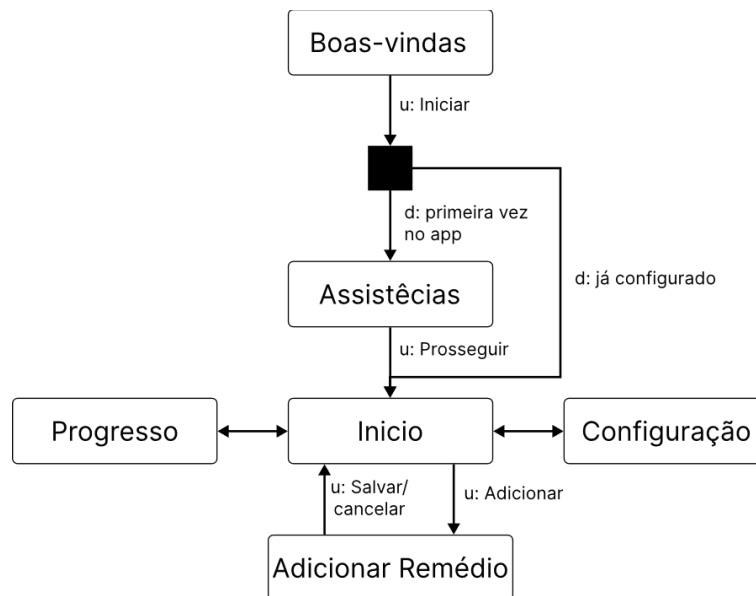


Fonte: Autor, 2026.

A navegação do usuário foi concebida de forma minimalista, priorizando a simplicidade e a redução da quantidade de telas, a fim de facilitar o uso, especialmente por usuários com menor familiaridade com dispositivos móveis. Dessa forma, foram definidas as seguintes telas: uma tela de boas-vindas, uma tela de configuração de assistência, três telas principais (início, progresso e configurações), uma tela destinada ao cadastro de medicamentos e uma

tela para o exportação dos horários. Na Figura 11, é apresentado um diagrama ilustrando a sequência de uso das telas e a interação do usuário com a aplicação.

Figura 11 - Sequência de uso de telas



Fonte: Autor, 2026.

A tela de boas-vindas, apresentada na Figura 12, é composta pela identidade visual do aplicativo (logotipo) e por um acesso direto aos termos de uso, permitindo que o usuário consulte as condições de utilização desde o primeiro contato com a aplicação.

Na Figura 13, é apresentada a tela de configuração de assistências, na qual o usuário pode selecionar os tipos de suporte oferecidos pelo aplicativo, adequando a experiência às suas necessidades, especialmente no que se refere à acessibilidade.

A tela inicial tem como principal função exibir os medicamentos cadastrados, juntamente com seus respectivos horários de consumo, permitindo ao usuário acompanhar de forma clara sua rotina de medicação. Essa tela é apresentada na Figura 14, onde há também um botão para adição de novos medicamentos, que direciona o usuário para a tela de “Adicionar remédio”, apresentada na Figura 15.

Por padrão, a aplicação realiza uma consulta à API para obter todos os medicamentos disponíveis, juntamente com suas respectivas doses cadastradas. Durante o processo de adição de um medicamento, o usuário deve informar a dose desejada, podendo selecionar uma dose existente ou cadastrar uma nova caso ela ainda não esteja disponível. Além disso, devem ser definidos a quantidade de comprimidos, os horários para administração e as

imagens associadas, que serão posteriormente apresentadas na tela de alarme durante o momento de lembrete do medicamento.

A aplicação disponibiliza três métodos para a definição dos horários de consumo dos medicamentos: intervalo, manual e mensal. No método por intervalo, o usuário informa um horário inicial de consumo e a quantidade de doses diárias desejadas. Com base nessas informações, a aplicação realiza o cálculo automático dos demais horários de administração. No método manual, o usuário define individualmente cada horário de consumo do medicamento. Por fim, no método mensal, o usuário seleciona um dia específico do mês e informa os horários de consumo associados àquela data.

Para a adição de imagens aos medicamentos, os usuários podem selecionar imagens previamente armazenadas no servidor e disponibilizadas publicamente, conforme apresentada na Figura 16. Caso não seja encontrada uma imagem compatível com o medicamento desejado, o usuário poderá realizar o envio de uma imagem a partir da galeria do dispositivo. Além disso, é disponibilizada a opção de compartilhar essa imagem com a API, permitindo que ela seja disponibilizada para utilização por futuros usuários.

A aplicação registra todos os agendamentos em uma memória própria, além de integrá-los ao sistema de notificações do dispositivo. Dessa forma, não é necessário que a aplicação permaneça em execução em segundo plano para o funcionamento dos alertas. Os agendamentos podem ser visualizados na tela inicial (Figura 14), juntamente com seus respectivos status de consumo, sendo classificados como futuro para medicamentos ainda não administrados e tomado para aqueles cujo consumo já foi realizado.

O sistema de notificação da aplicação é implementado por meio da abertura automática de uma tela específica, associada a um sistema de síntese de voz para auxiliar o usuário durante o processo de consumo. A Figura 17 apresenta a interface de notificação, na qual são exibidas as informações referentes ao medicamento a ser administrado, incluindo nome, dosagem, quantidade e imagens associadas ao remédio.

Para tornar o processo de acompanhamento mais eficiente, medicamentos com horários de consumo próximos, considerando uma diferença de até 30 minutos entre os agendamentos, são agrupados em uma única notificação. Dessa forma, o usuário pode visualizar e confirmar o consumo de múltiplos medicamentos em uma única interação, reduzindo a quantidade de alertas recebidos.

Após a confirmação do consumo dos medicamentos, o usuário é direcionado para uma tela de conclusão e incentivo (Figura 18), na qual é apresentada a pontuação obtida. O cálculo da pontuação considera a quantidade de medicamentos consumidos e a sequência de dias consecutivos em que o tratamento foi seguido corretamente. Cada medicamento consumido corresponde a um ponto, enquanto a manutenção de uma sequência de consumo sem interrupções adiciona três pontos extras ao total obtido.

Todo o progresso do usuário é armazenado localmente no dispositivo, sem qualquer compartilhamento de dados com a rede. As informações referentes ao histórico de interações, pontuação obtida e proporções de consumo dos medicamentos são organizadas e apresentadas na página de progresso (Figura 19), permitindo que o usuário acompanhe seu desempenho e adesão ao tratamento ao longo do tempo.

Apesar de a aplicação atender diferentes perfis de usuários e necessidades, considera-se que nem todos os usuários possuirão um dispositivo móvel compatível ou terão acesso contínuo ao aplicativo. Dessa forma, foi implementado um sistema de exportação dos medicamentos agendados no formato PDF, com suporte à impressão. Essa funcionalidade pode ser acessada por meio do ícone de impressora localizado no canto superior direito da aplicação, permitindo a geração de uma lista física dos medicamentos a serem consumidos. Na Figura 20 é apresentada a página de exportação de arquivos, contendo um exemplo de relatório de horários de consumo para o usuário “Paciente”, gerado e exportado pela “Assistente”. Os nomes dos usuários são configurados previamente à exportação do arquivo, permitindo a personalização do documento conforme o contexto de uso.

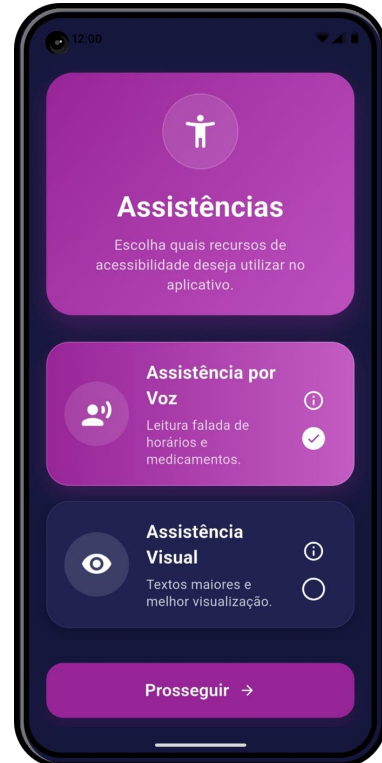
Por fim, todos os parâmetros de operação da aplicação, com exceção do sistema de pontuação, podem ser configurados por meio da página de configurações (Figura 21). Nessa seção, o usuário também possui a opção de remover todos os registros armazenados pela aplicação, sendo necessária uma confirmação prévia para a execução dessa ação, garantindo a segurança e evitando exclusões acidentais.

Figura 12 - Tela de boas-vindas



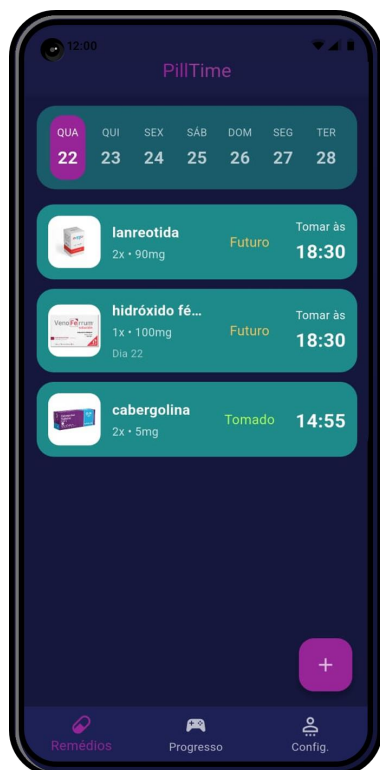
Fonte: Autor, 2026.

Figura 13 - Tela de assistências



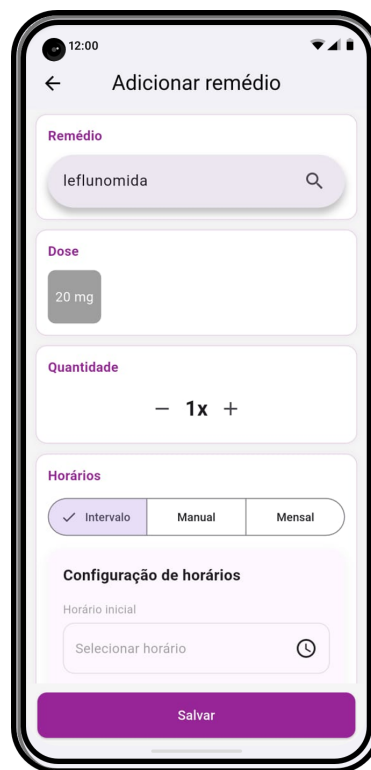
Fonte: Autor, 2026.

Figura 14 - Tela de início



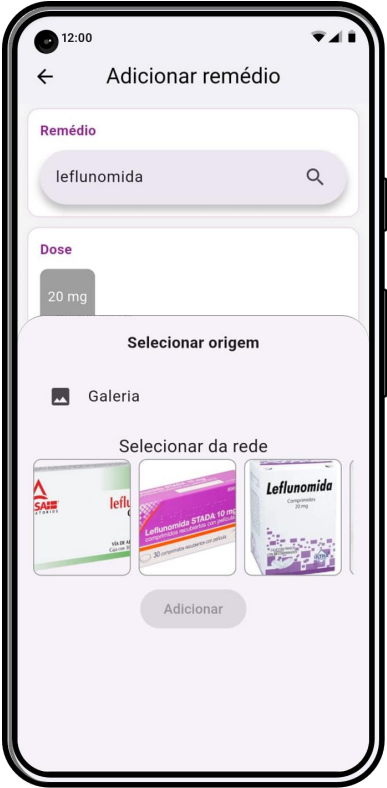
Fonte: Autor, 2026.

Figura 15 - Tela para adicionar remédios



Fonte: Autor, 2026.

Figura 16 - Modal para adicionar fotos



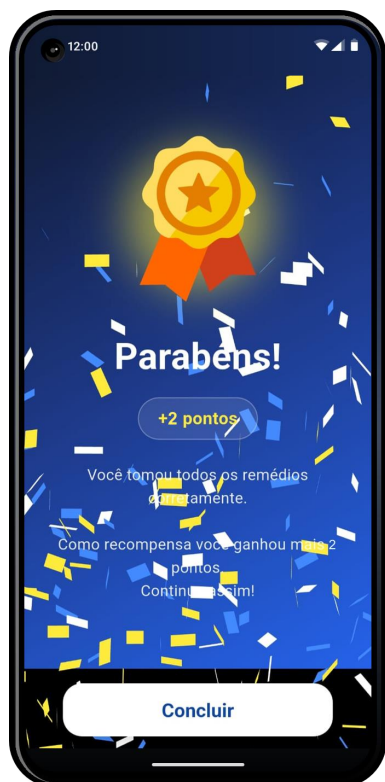
Fonte: Autor, 2026.

Figura 17 - Notificação



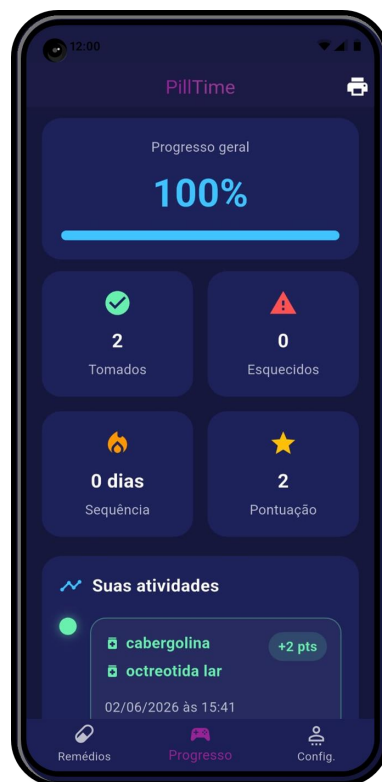
Fonte: Autor, 2026.

Figura 18 - Página de parabenização



Fonte: Autor, 2026.

Figura 19 - Tela de progresso



Fonte: Autor, 2026.

Figura 20 – Tela de exportação



Fonte: Autor, 2026.

Figura 21 - Tela de Configurações



Fonte: Autor, 2026.

CRONOGRAMA DE ATIVIDADES

As atividades a serem desenvolvidas no projeto são apresentadas no Quadro 1.

Quadro 1 - Exemplo de cronograma de atividades a serem desenvolvidas no projeto

Atividades/mês	Março	Abil	Maio	Junho	Julho
Levantar requisitos					
Elencar ferramentas de desenvolvimento					
Modelar base de dados relacional					
Popular a base de dados					
Desenvolver aplicação móvel					
Desenvolver API central					
Testar sistema com usuários reais					
Entrega final					

RESULTADOS ESPERADOS

Espera-se, ao final da execução do projeto, o desenvolvimento de uma aplicação mobile funcional, acessível e intuitiva, capaz de auxiliar usuários no gerenciamento adequado do uso de medicamentos.

Do ponto de vista técnico, espera-se a implementação de uma solução estruturada em camadas, composta por aplicação *mobile*, *API REST* e banco de dados relacional. O sistema deverá apresentar bom desempenho mesmo em dispositivos com recursos limitados, fazendo o armazenamento apenas dos remédios utilizados pelo usuário, fazendo pouco uso de memória local.

Adicionalmente, espera-se que os mecanismos de gamificação incorporados à aplicação incentivem o engajamento dos usuários no cumprimento correto de seus tratamentos, promovendo maior adesão às rotinas de medicação por meio de recompensas, metas e acompanhamento de progresso.

No âmbito social, o projeto visa contribuir para a melhoria da qualidade de vida dos usuários, especialmente aqueles com dificuldades de leitura ou organização, promovendo maior autonomia e segurança no cuidado com a saúde. Espera-se também fortalecer a inclusão digital ao oferecer uma ferramenta simples e acessível.

Por fim, no contexto acadêmico, o projeto contribuirá para a formação do discente por meio da aplicação prática de conhecimentos adquiridos ao longo do curso, integrando aspectos de desenvolvimento de software, banco de dados, usabilidade e acessibilidade, além de reforçar o papel da universidade como agente de transformação social.

REFERÊNCIAS

FAMURS. REMUME - Famurs. Disponível em: <<https://famurs.com.br/remume#inicioRemume>>. Acesso em: 17 jun. 2026.

SECRETARIA DA SAÚDE, Rio Grande do Sul. Lista de medicamentos e fórmulas nutricionais na Farmácia Digital RS 2025. [S.l.]: Secretaria da Saúde do Estado do Rio Grande do Sul, 2025. Disponível em: <<https://admin.saude.rs.gov.br/upload/arquivos/202508/22100532-lista-de-medicamentos-e-formulas-nutricionais-na-farmacia-digital-rs-2025-08-22.pdf>>.